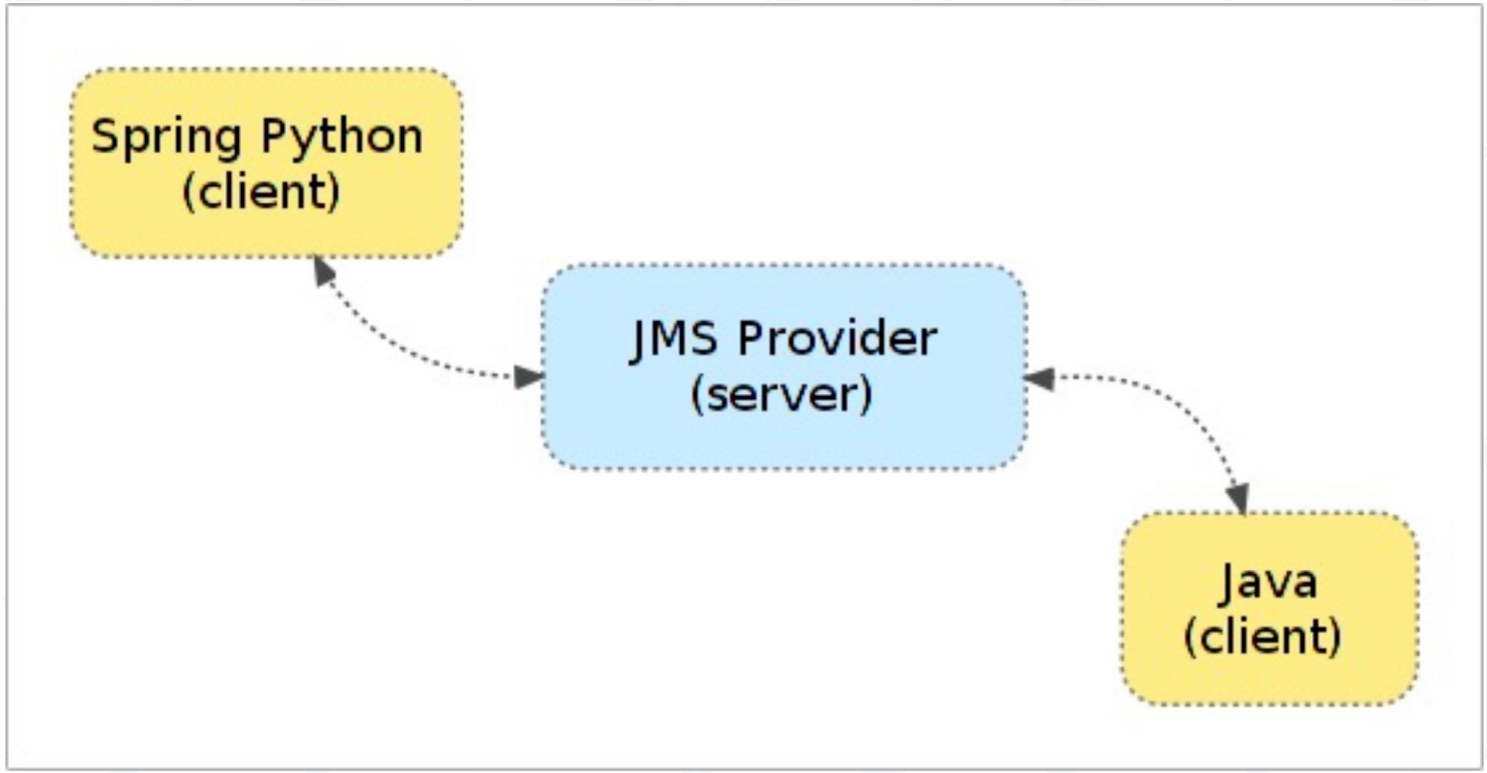
Lesson 36

04.05.2026

Java Message Service

Java Message Service (JMS) - стандарт проміжного ПЗ для розсилки повідомлень, що дозволяє програмам створювати, надсилати, отримувати та читати повідомлення.



Комунікація між компонентами, що використовують JMS, асинхронна (процедура не очікує відповіді на своє повідомлення) та незалежна від виконання компонентів. JMS підтримує дві моделі обміну повідомленнями: «точка – точка» та «видавець передплатник».

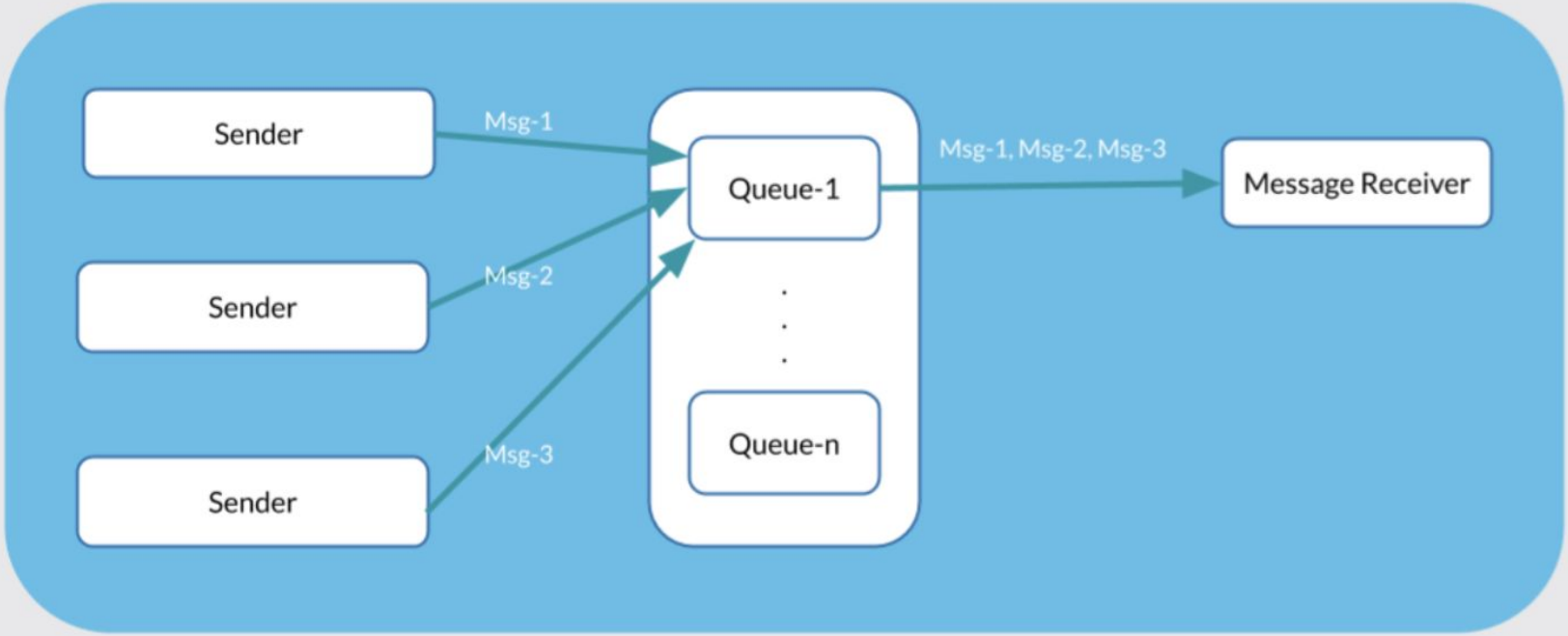
Модель «точка - точка» характеризується наступним:

- Кожне повідомлення має лише одного адресата
- Повідомлення потрапляє в «чергу» адресата і може бути прочитане будь-коли. Якщо адресат не працював у момент надсилання повідомлення, повідомлення непропаде.
- Після отримання повідомлення адресат надсилає повідомлення.

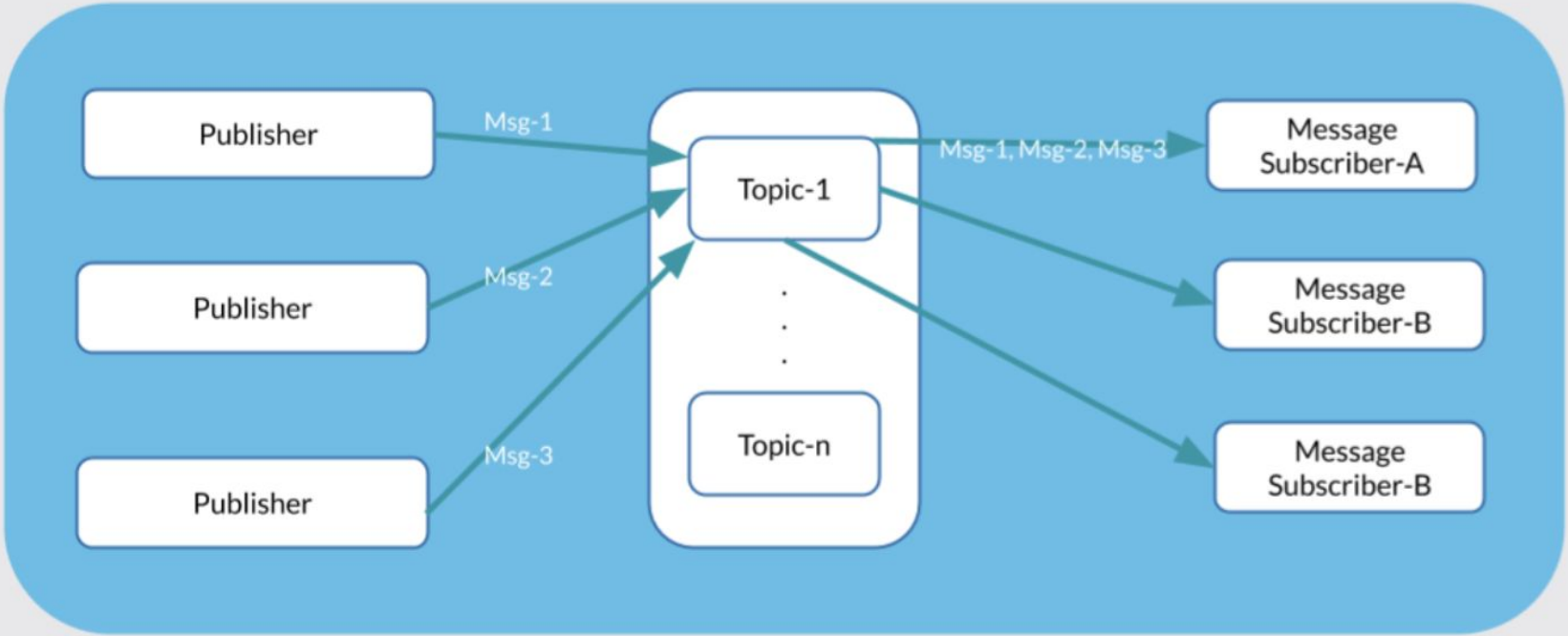
Модель «видавець-передплатник» характеризується таким:

- Передплатник підписується на певну «тему»
- Видавець публікує своє повідомлення. Його одержують усі передплатники цієї теми
- Одержувач повинен працювати і бути підписаний у момент відправлення повідомлення

JMS Point to Point Messaging



JMS Pub-Sub Model



kafka

redis

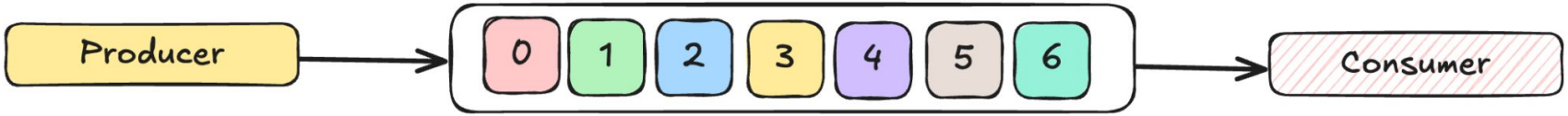
RabbitMQ

Apache
ACTIVEMQ

RabbitMQ

RabbitMQ

RabbitMQ – це брокер повідомлень. Його основна мета – приймати та віддавати повідомлення. Його можна уявляти, як поштове відділення: коли Ви кидаєте лист у скриньку, Ви можете бути впевнені, що рано чи пізно листоноша доставить його адресату. У цій аналогії RabbitMQ є одночасно і поштовою скринькою, і поштовим відділенням, і листоношою.



Producer (постачальник) – програма, яка надсилає повідомлення. У схемах він буде представлений кругом із літерою «P».

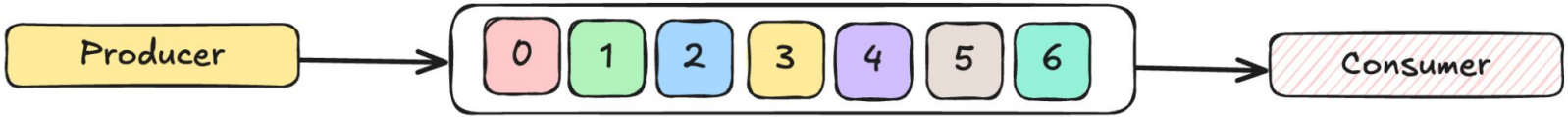
Queue (черга) – ім'я «поштової скриньки». Вона існує усередині RabbitMQ. Хоча повідомлення проходять через RabbitMQ та програми, вони зберігаються тільки в чергах. Черга не має обмежень на кількість повідомлень, вона може прийняти скільки завгодно велику їх кількість – можна вважати її нескінченною буфер. Будь-яка кількість постачальників може надсилати повідомлення в одну чергу, також будь-яка кількість передплатників може отримувати повідомлення з однієї черги. У схемах черга буде позначена стеком та підписана ім'ям (HELLO).

Consumer (слухач) – програма, яка приймає повідомлення. Зазвичай передплатник перебуває у стані очікування повідомлень. У схемах він буде представлений кругом із літерою «C»:

Hello World!

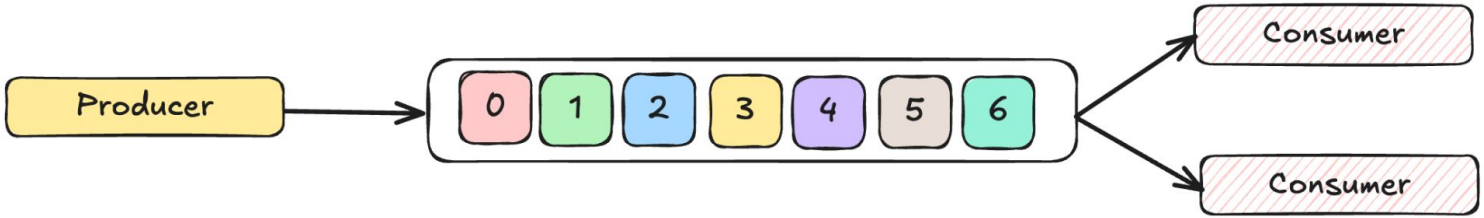
Перший приклад не буде особливо складним – давайте просто відправимо повідомлення, приймемо його та виведемо на екран. Для цього нам знадобиться дві програми: одна буде надсилати повідомлення, інша – приймати та виводити їх на екран.

Загальна схема така:



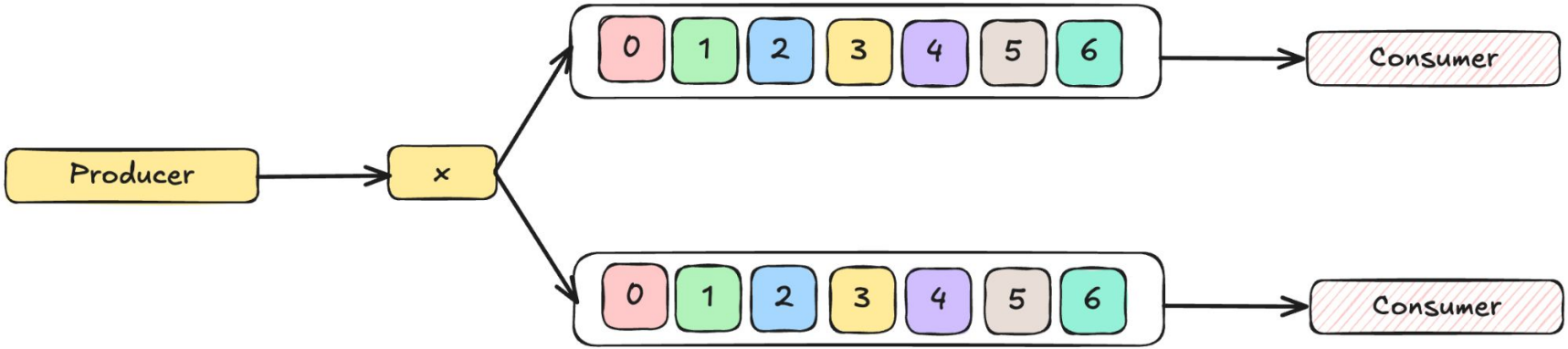
Work Queues

В даному прикладі одну чергу слухають вже два листенери. Для емуляції корисної роботи використовуємо Thread.sleep. Важливо, що листенери однієї черги можуть і на різних інстансах програми. Так можна розпаралелити чергу на кілька комп'ютерів або нод у хмарі.

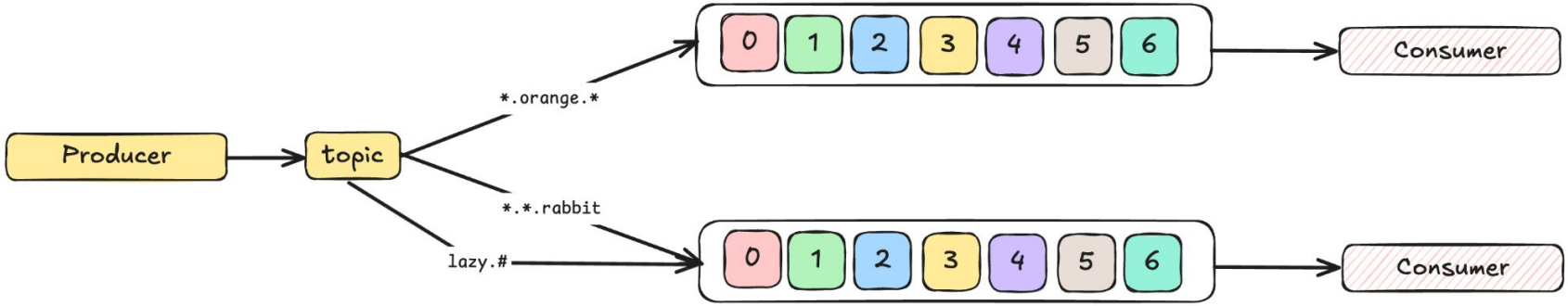


Publish/Subscribe

Тут те саме повідомлення надходить відразу двом консьюмерам.

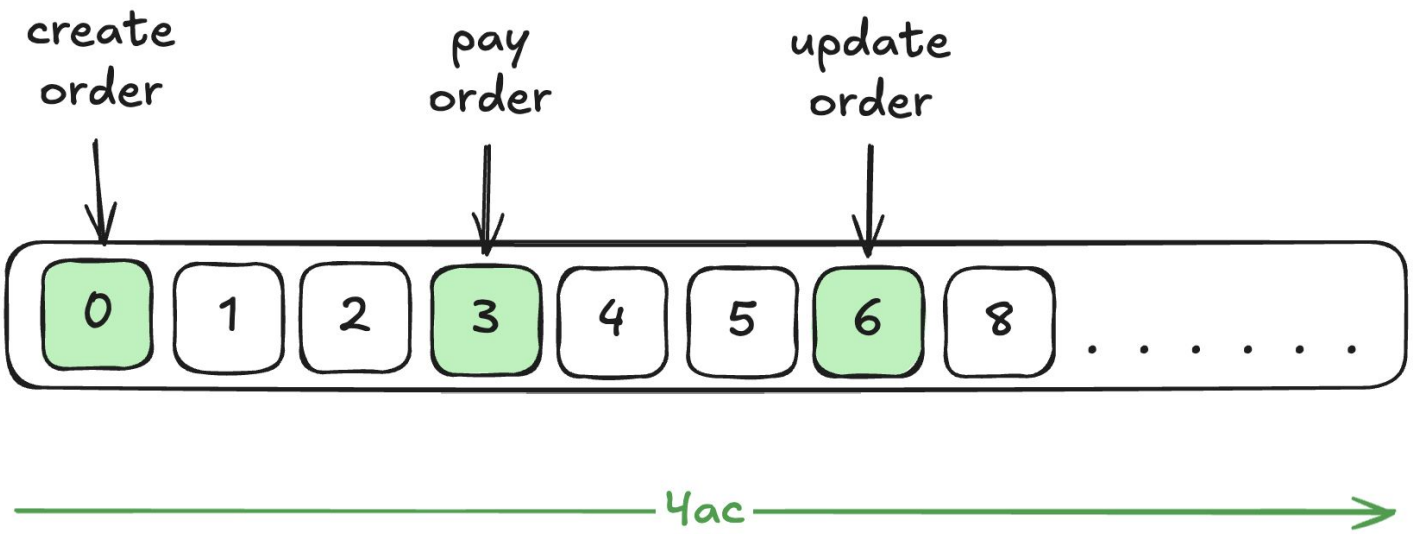


Topics


Producer
topic
.orange.
..rabbit
lazy.#
0
1
2
3
4
5
6
Consumer
0
1
2
3
4
5
6
Consumer

Apache Kafka — розподілена система обміну повідомленнями між серверними застосунками в режимі реального часу. Завдяки високій пропускній здатності, масштабованості й надійності використовується в компаніях, які працюють з великими обсягами даних. Написана мовами Java та Scala.

Лог — це впорядкований потік подій у часі. Подія відбувається, потрапляє в кінець лога і залишається там незмінною.

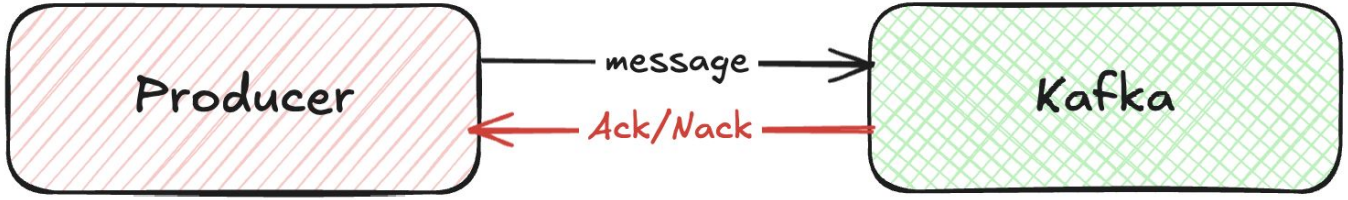


Apache Kafka керує логами та організовує платформу, яка з’єднує постачальників даних із споживачами та дає можливість отримувати впорядкований потік подій у реальному часі.

Продюсери

Для запису подій у кластер Kafka існують продюсери — це застосунки, які ви розробляєте.

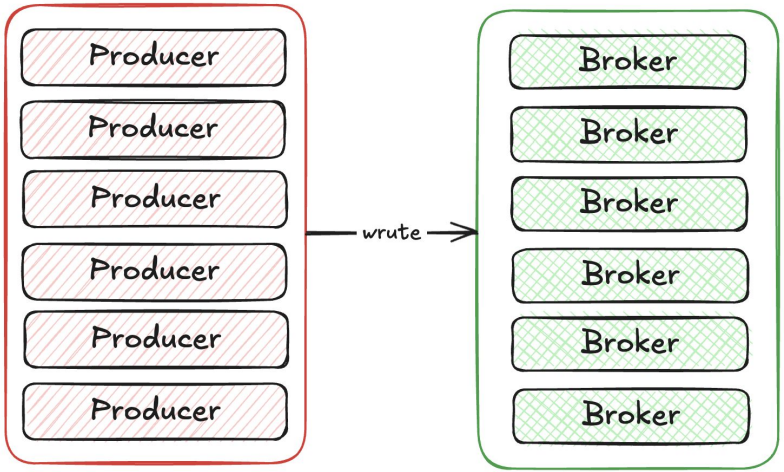
Програма-продюсер записує повідомлення в Kafka, той зберігає події й повертає підтвердження про запис (acknowledgement). Продюсер отримує це підтвердження та починає наступний запис.



Брокери

Кластер Kafka складається з брокерів. Можна уявити систему як дата-центр і сервери в ньому. Під час першого знайомства думайте про Kafka-брокер як про комп’ютер: це процес в операційній системі з доступом до свого локального диска.

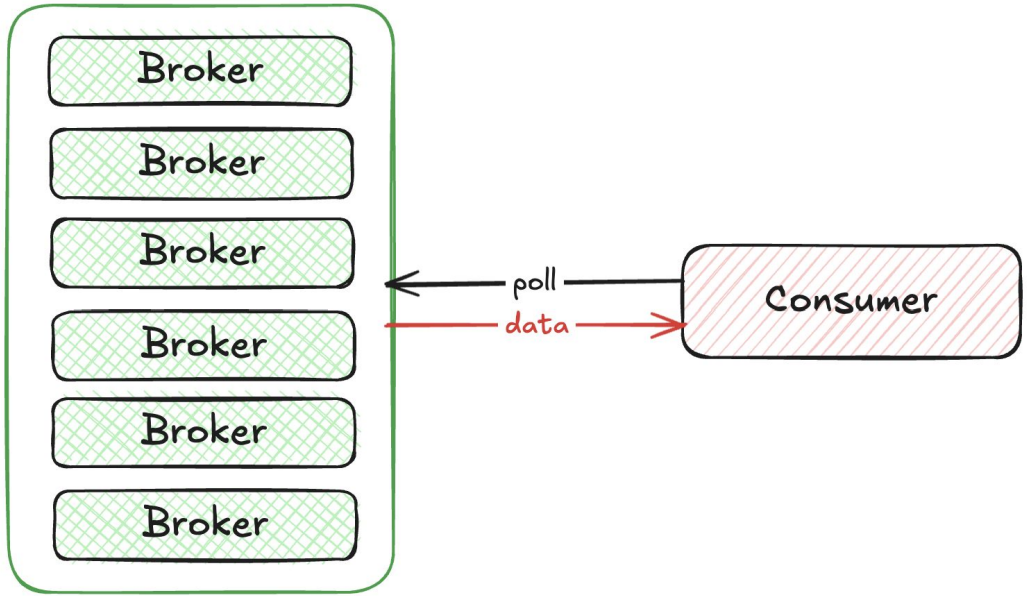
Усі брокери з’єднані один з одним мережею та діють спільно, утворюючи єдиний кластер. Коли ми говоримо, що продюсери записують події в Kafka-кластер, то маємо на увазі, що вони працюють із брокерами в його складі.



Споживачі (консумери)

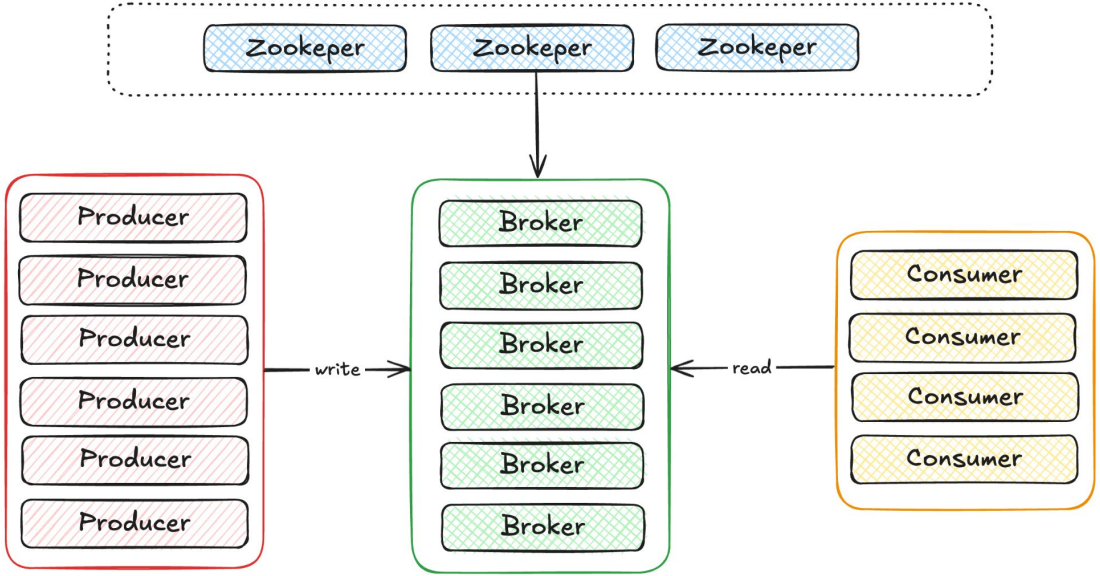
Події, які продюсери записують на локальні диски брокерів, можуть читати споживачі — це також застосунки, які ви розробляєте. У цьому випадку кластер Kafka все ще є чимось, що обслуговується інфраструктурою, а ви як користувач пишете продюсерів і консумерів.

Програма-споживач підписується на події або проводить опитування (polling) і отримує дані у відповідь. Так процес триває по колу.



Архітектура Kafka

Розгляньмо архітектуру Kafka уважніше. Ліворуч є продюсери, посередині — брокери, праворуч — консумери. Інструмент являє собою групу брокерів, пов'язаних із кворумом ZooKeeper. Kafka використовує ZooKeeper для досягнення консенсусу щодо стану в розподіленій системі: є кілька речей, які мають бути узгоджені кожним брокером, і ZooKeeper допомагає досягти цього «узгодження» всередині кластера.



ZooKeeper — це виділений кластер серверів для формування кворуму-узгодження та підтримки внутрішніх процесів Kafka. Завдяки цьому інструменту ми можемо керувати кластером Kafka: додавати користувачів і топіки, задавати їм налаштування.

01 керування кластером

02 виявлення збою та відновлення

03 зберігання ACL та секретів

Також ZooKeeper допомагає виявити збої контролера, обрати нового та зберегти працездатність кластера Kafka. Крім того, він зберігає всі авторизаційні дані та обмеження, або ж Access Control Lists, під час роботи споживачів і продюсерів із брокерами.

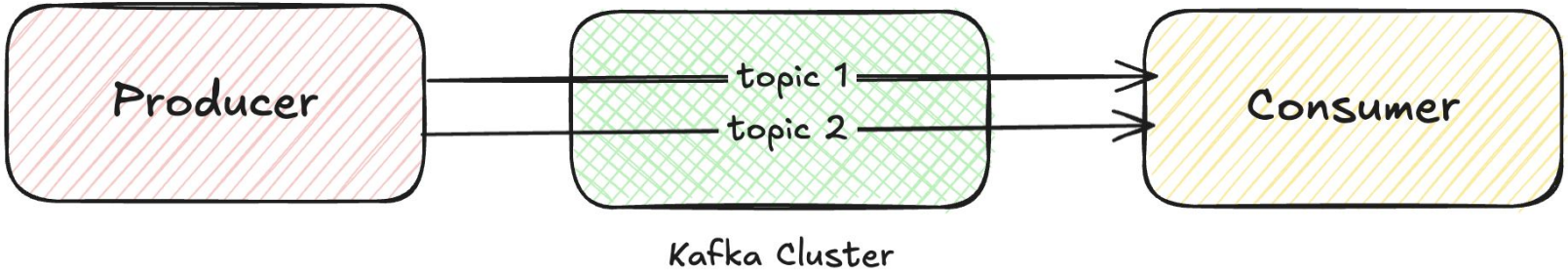
Топіки

Топік — це логічне розділення категорій повідомлень на групи. Наприклад, події за статусами замовлень, координатами партнерів, маршрутними листами тощо.

Ключове слово тут — *логічне*. Ми створюємо топіки для подій однієї групи та намагаємося не змішувати їх між собою. Наприклад, координати партнерів не повинні бути в тому ж топіку, що й статуси замовлень, а оновлені статуси по замовленнях не варто складати вперемішку з оновленням реєстрацій користувачів.

Топік зручно уявляти як лог: ви записуєте подію в кінець і не руйнуєте при цьому ланцюжок старих подій. Загальна логіка:

- Один продюсер може писати в один або кілька топіків
- Один консумер може читати один або кілька топіків
- В один топік можуть писати один або більше продюсерів
- З одного топіка можуть читати один або більше консумерів

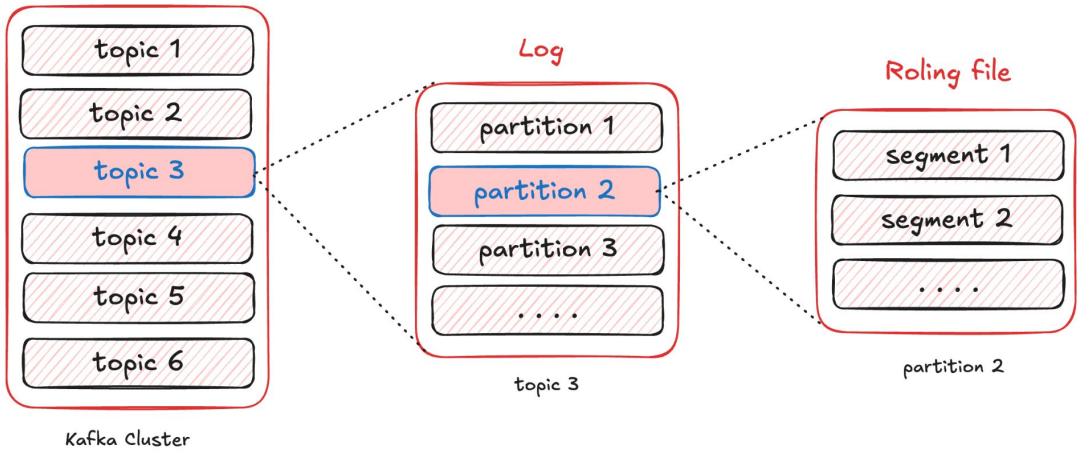


Партиції та сегменти

Топіків у Kafka може бути скільки завгодно, але продуктивність обмежена ресурсами машин. Щоб масштабувати систему, топік ділять на партиції. Кожен топік складається з однієї або кількох партицій, які можна розміщувати на різних брокерах. Саме завдяки цьому Kafka горизонтально масштабується.

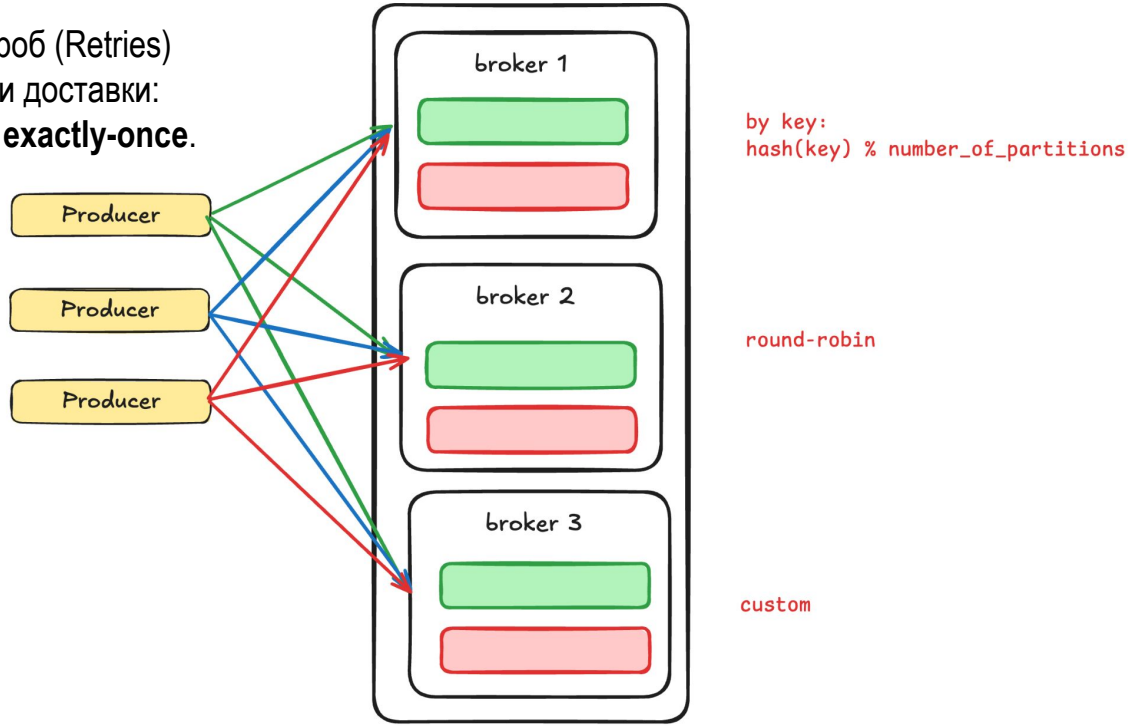
Партиція — це впорядкований лог подій: нові повідомлення додаються тільки в кінець та ніколи не змінюються. Глобального порядку всередині топіка немає, але є порядок у межах кожної партиції.

Фізично партиції складаються із сегментів — окремих файлів на диску.

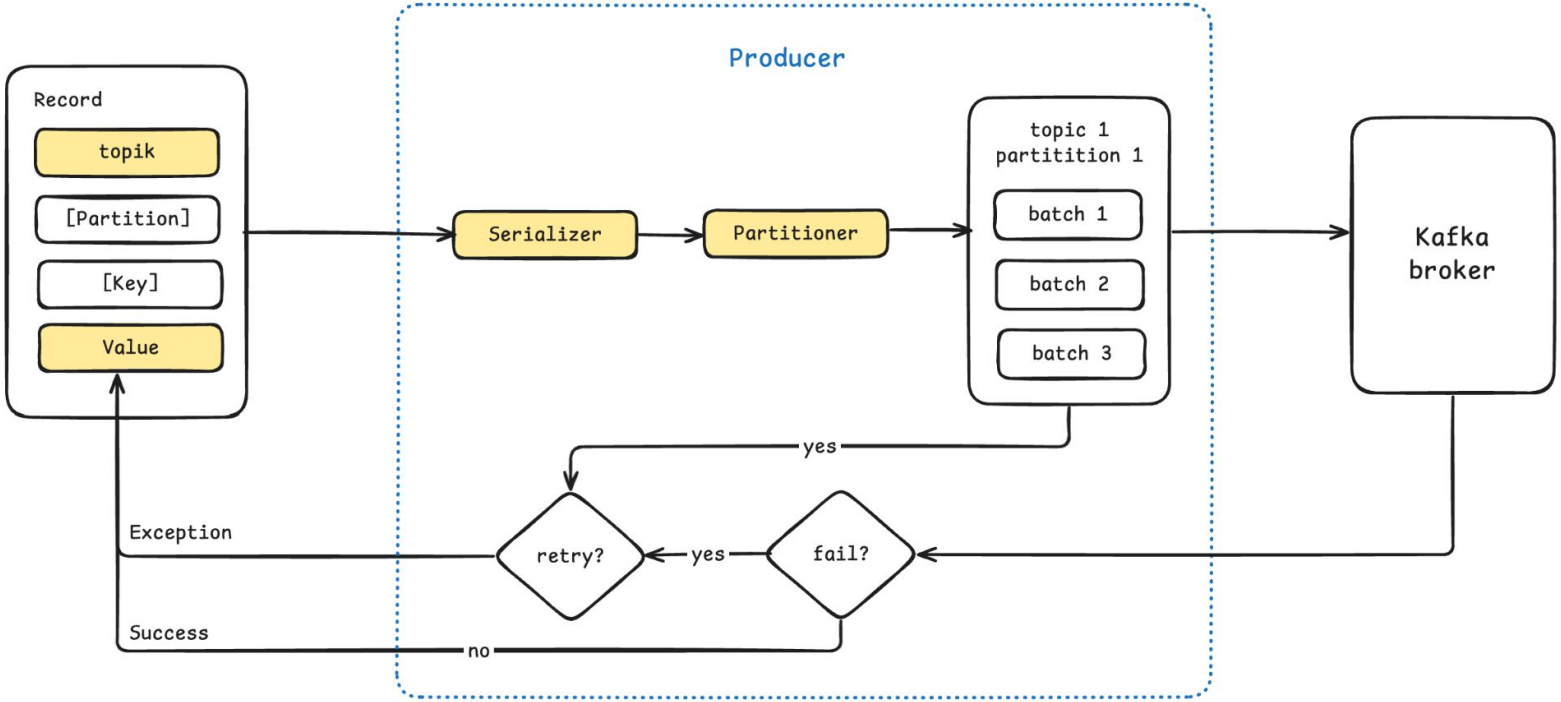


Kafka Producer

- Продюсери самостійно партицюють повідомлення та задають алгоритм партиціювання (round-robin, hash-based або кастомний).
- **Гарантований порядок повідомлень існує лише в межах однієї партиції.**
- Продюсер обирає:
  - розмір батчу
    кількість повторних спроб (Retries)
- Kafka підтримує всі семантики доставки:
  at-most once, **at-least once**, **exactly-once**.



1. Пейлоад пакується у структуру з вказанням: топіка, партиції, ключа партиціювання.
2. Пейлоад серіалізується у формат:JSON, Protobuf, Avro, власний формат із підтримкою схем.
3. Визначається партиція відповідно до переданого ключа та обраного алгоритму.
4. Повідомлення групуються у батчі визначеного розміру.
5. Батчі надсилаються брокеру Kafka для збереження.



Семантики доставки повідомлень

У будь-яких чергах існує компроміс між швидкістю та надійністю.

At-most once

Втрата повідомлень допускається

Дублікатів бути не повинно

Найслабша гарантія, реалізована більшістю брокерів

At-least once

Повідомлення не повинні губитися

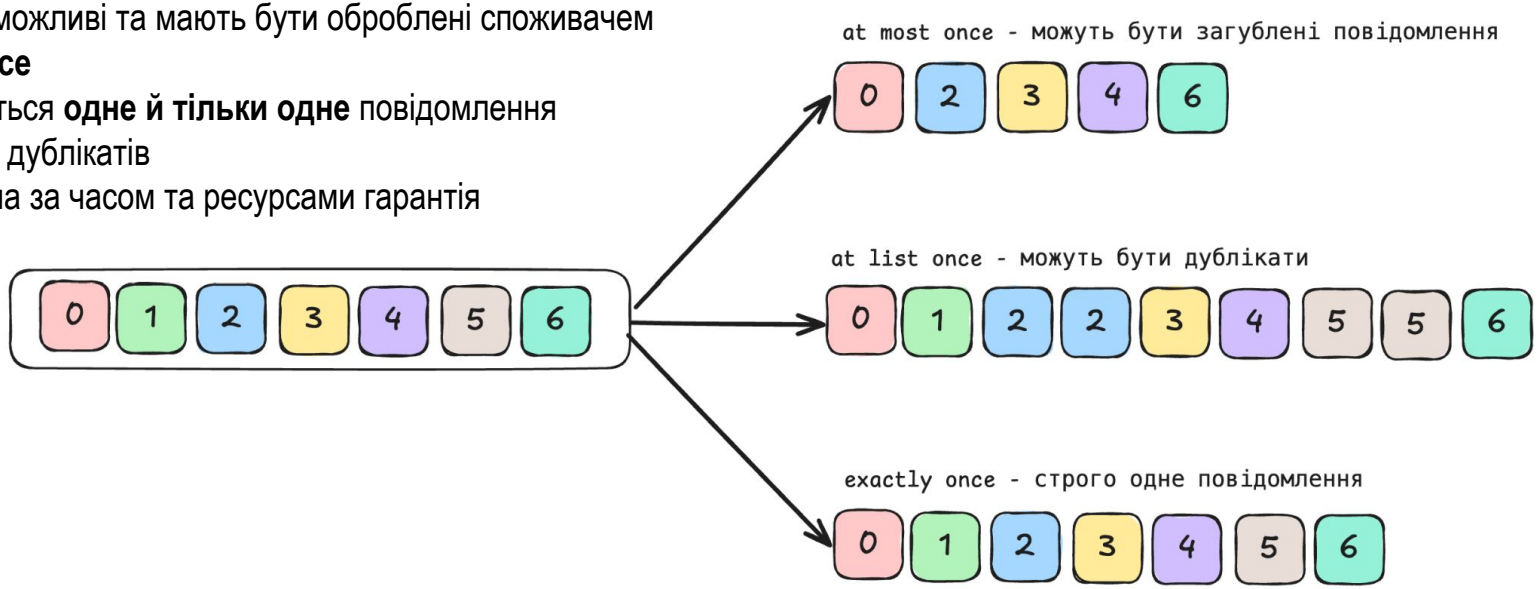
Дублікати можливі та мають бути оброблені споживачем

Exactly-once

Доставляється **одне й тільки одне** повідомлення

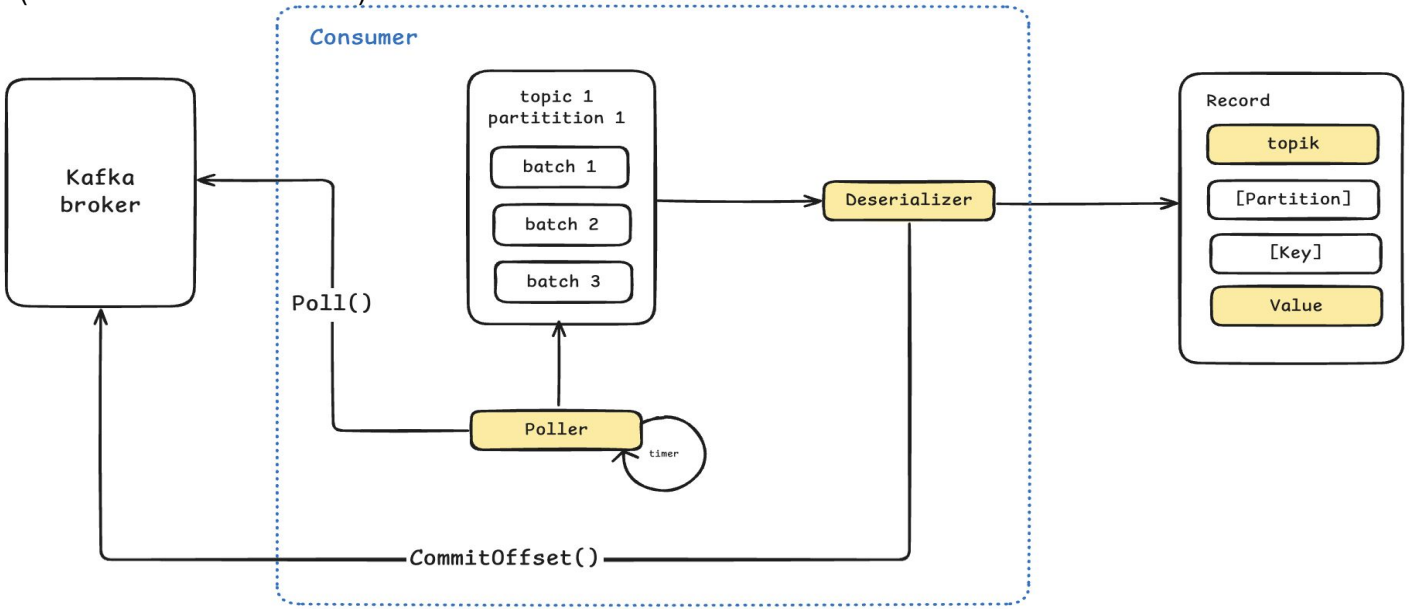
Ні втрат, ні дублікатів

Найдорожча за часом та ресурсами гарантія



Kafka Consumer

Після запуску консюмер **періодично poll-ить** нові записи з брокера
Поллер отримує **batch-и** повідомлень по топіках і партиціях
Далі відбувається: **десеріалізація** повідомлень, **обробка** (бізнес-логіка)
Після опрацювання консюмер може: **закомітити офсет** — зафіксувати позицію та читати далі, комітити **синхронно** або **асинхронно** або **не комітити** зовсім
Якщо консюмер обробив дані, але **аварійно впав** і не закомітив офсет: наступний консюмер **повторно прочитає** ті ж повідомлення (semantics: *at-least once*)

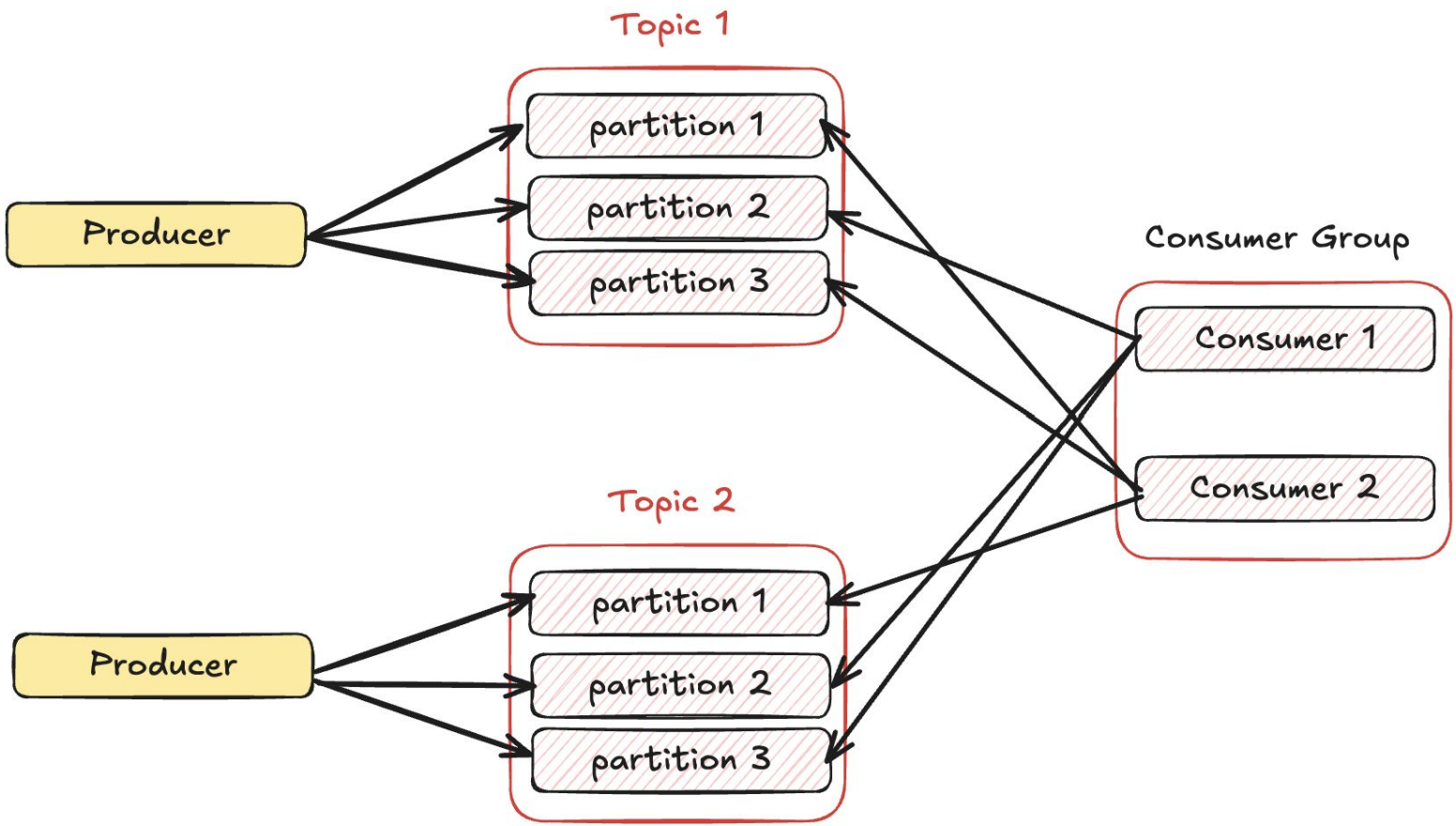


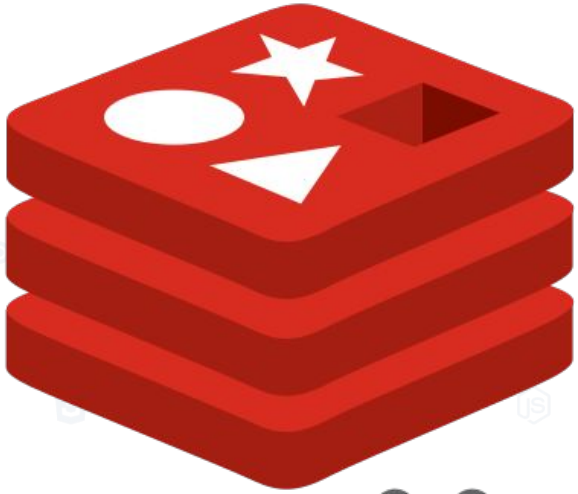
Консюмер-групи в Kafka

- Якщо всі партиції читав би один консюмер — це **повільно та немасштабовано,** тому консюмери можуть об’єднуватись у **консюмер-групи**
- Розподіл партицій:
  відбувається **автоматично** на стороні брокера
  Kafka намагається розподілити максимально **справедливо**
  один консюмер **не може** читати одну й ту саму партицію з іншим
- У кожної консюмер-групи:
  є унікальний **group.id**
  простір імен груп **глобальний** для всього кластера

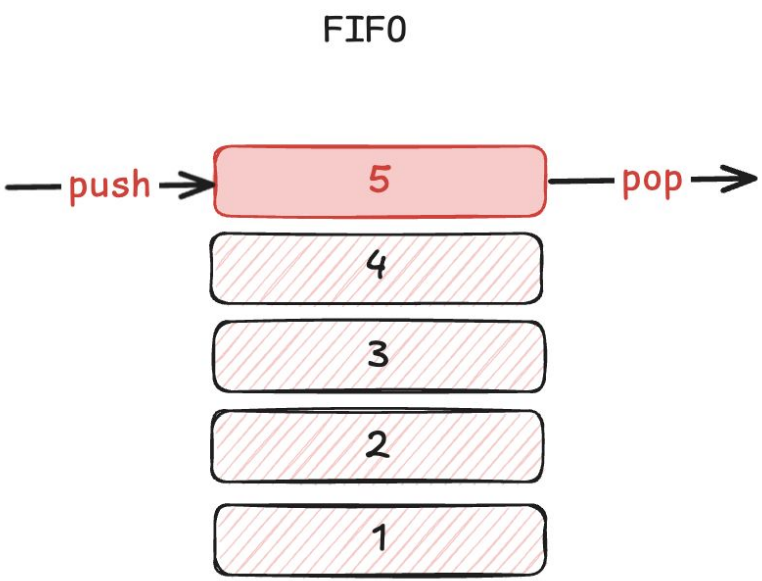
Головне:
Kafka зберігає **офсети** партицій самостійно при відключенні / перезапуску консюмера читання продовжиться **з останнього офсету**

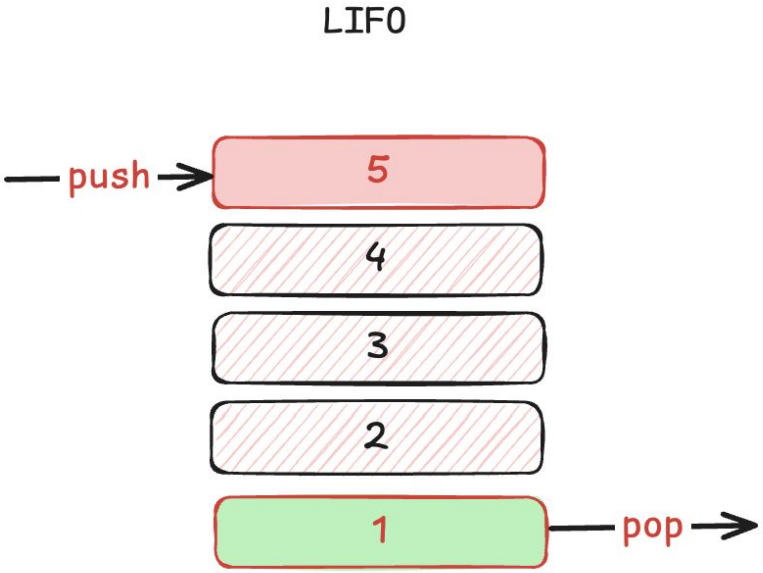
Коміти офсету:
відправляються після обробки повідомлень
можна комітити до обробки, але частіше це **небезпечно**
завдяки цьому консюмер-групи ідеальні для **event-driven** систем та безболісних деплоїв

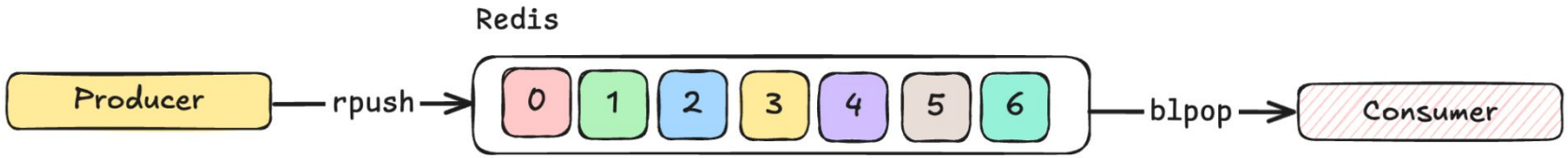

Topic 1
partition 1
partition 2
partition 3
Producer
Consumer Group
Consumer 1
Consumer 2
Topic 2
partition 1
partition 2
partition 3
Producer



redis


FIFO
push
5
pop
4
3
2
1


LIFO
push
5
4
3
2
1
pop


Redis
Producer
rpush
0
1
2
3
4
5
6
blpop
Consumer